

Project 1:

Customer Churn Prediction

Problem Definition

Customer churn refers to customers leaving a service or platform. Predicting churn is crucial for businesses to improve retention and reduce revenue loss.

The challenge is to develop a machine learning system that can accurately predict whether a customer will churn based on behavioral and demographic data.

Project Description

This project implements a machine learning-based churn prediction system using customer-related features such as income, spending behavior, engagement, and purchase history.

The workflow includes:

1. Training multiple supervised learning models
2. Evaluating their performance using accuracy
3. Selecting the best-performing model
4. Deploying the final model using a Streamlit application

After comparison, Decision Tree Classifier achieved the best performance and was selected as the final model.

The system also provides probability scores, giving insights into how likely a customer is to churn.

Tools & Libraries Used

Programming Language:

- Python

Libraries:

- **Streamlit** → Web application UI
- **NumPy** → Numerical operations
- **Scikit-learn** → Model training & evaluation
- **Pickle** → Model and scaler storage

Code Implementation:

```
import streamlit as st

import numpy as np

import pickle


# LOAD FILES

model = pickle.load(open("churn_model.pkl", "rb"))

scaler = pickle.load(open("scaler.pkl", "rb"))

features = pickle.load(open("features.pkl", "rb"))


# PAGE CONFIG

st.set_page_config(page_title="Churn Prediction", layout="centered")


# TITLE

st.markdown("<h1 style='text-align: center; color: #4CAF50;'>Customer Churn Prediction</h1>",
unsafe_allow_html=True)

st.markdown("<p style='text-align: center;'>AI-powered prediction of customer churn</p>",
unsafe_allow_html=True)


st.divider()


# INPUTS

st.subheader("Enter Customer Details")


col1, col2 = st.columns(2)


with col1:

    income = st.slider("Income", 1000, 100000, 30000)

    review = st.slider("Review Score", 1, 5, 3)

    purchase = st.slider("Purchase Amount", 100, 50000, 5000)

    session = st.slider("Session Time", 1, 300, 60)
```

```
with col2:

    spending = st.slider("Spending Score", 1, 100, 50)
    age = st.slider("Age", 18, 100, 30)
    returns = st.slider("Returns", 0, 50, 5)
    days = st.slider("Days Since Last Purchase", 1, 365, 30)
```

```
st.divider()
```

```
# PREDICTION
```

```
if st.button("Predict Churn", use_container_width=True):
```

```
    # Map inputs correctly
```

```
    input_dict = {
```

```
        'Income': income,
```

```
        'ReviewScore': review,
```

```
        'PurchaseAmount': purchase,
```

```
        'SessionTime': session,
```

```
        'SpendingScore': spending,
```

```
        'Age': age,
```

```
        'Returns': returns,
```

```
        'days_since_purchase': days
```

```
    }
```

```
    # Maintain correct order
```

```
    input_data = np.array([[input_dict[col] for col in features]])
```

```
    # Scale
```

```
    input_scaled = scaler.transform(input_data)
```

```
    # Predict
```

```
    prediction = model.predict(input_scaled)
```

```

prob = model.predict_proba(input_scaled)[0][1]

# Output
st.subheader("Prediction Result")

if prediction[0] == 1:
    st.error(f"⚠️ Customer is likely to CHURN\n\nProbability: {prob:.2f}")
else:
    st.success(f"✅ Customer will STAY\n\nProbability: {prob:.2f}")

st.progress(int(prob * 100))

```

Output:





